

One Imperfect Page per Hundred: Testing Scribe on Data I Didn't Curate

When I tested my pipeline on data outside its own benchmark, brand-name reading accuracy fell from roughly 80% to **38%**.

The share of pages flagged for human review rose from 28% to **97%**.

And exactly **one imperfect page in a hundred** reached the record without a person looking at it.

This post explains why I consider that a reasonable result.

Where this started

In [part 1](#), I built Scribe — a local pipeline that turns handwritten clinical forms into structured data and, more importantly, learned when *not* to trust a model's answer. The headline was that model confidence carried no signal, and disagreement between two independent reads did.

But every number in part 1 had a limitation.

I curated the gold answers. I wrote the extraction schema. I tuned the prompts against the same 23 pages I was scoring on. Even done carefully, that means the system and the test were developed together.

So I wanted a test set I had no hand in.

The external test

I found a [public dataset of 100 handwritten Indian prescriptions](#), where the ground truth — the list of medicines on each page — was written by the dataset's author, not by me.

The rules I set for myself:

- the extraction schema gets written once, from the task description, before seeing results
- the prompts stay frozen exactly as they were in part 1
- the author's answer conventions are the answer key, not mine

Two things make this set difficult. The handwriting is real doctors' cursive — not the careful volunteer writing my benchmark was built from. And the drug names are Indian brand names the models have seen little of.

That difficulty is useful: a system built for clinics in one country will eventually meet forms from another.

Reading accuracy dropped sharply

Brand names came back right 38% of the time.

CEPODEM became **Capelin**. *ESOTAB* became **Esotrib**. *NIVEOLI* became **Ulfah Niveoli**.

These are the same kind of confident misreads that affected patient names in part 1 — at roughly triple the rate, because the handwriting is harder and the vocabulary is unfamiliar.

I ran both of my best configurations, which differ only in which model does the second read. They produced **identical reading results** — same primary model, so the underlying reading was unchanged; the verifier only affected how much got flagged.

That repeats part 1's central lesson: **verification cannot fix reading. It can only catch it.**

No prompt I own fixes the reading either. For the local vision models I tested, hard cursive remains a major limitation — and a [recent benchmark on South African maternity records](#) reported similar difficulty even with frontier cloud models.

The review gate held

This is the part that mattered to me.

On these harder pages, the models were much less willing to claim certainty: most confidence scores fell to around 0.60–0.70, and **97–99% of pages were sent for human review**.

For this input, “a person must look at this” *is* the correct output.

Auditing the escapes

Three pages out of a hundred skipped human review. I checked every one by hand.

- Two were **fully correct** — single-medicine prescriptions, read cleanly.
- One was an eleven-medicine page that got ten of them right — and **silently dropped one** (BETNESOL INJ).

One imperfect page per hundred, on input considerably harder than anything in the development set.

Part 1 ended with a rule: *count what escapes, and you're grading the system*. This test is why that rule exists — it's what lets me report a specific audited failure count instead of a single accuracy number.

The test also caught a bug in my scorer

One more thing this exercise caught — in my own tooling.

My first scoring pass penalized extracted rows for containing dose schedules like (1-0-1), even when they named the right drug. The scorer's precision check compared token sets in the wrong direction. External data debugs your evaluator, not just your model.

What I'm taking away

- **Reading accuracy didn't transfer to new handwriting; the review-gate behavior did.** The parts that held up were the system parts — confidence behavior, dual reads, the review queue. The reading itself did not.
- **My production target got context.** Part 1's benchmark sits at 96% trusted-output accuracy; this set is too far from my actual target forms to measure the 98% goal against. The real number will land somewhere between the two, and only the held-out set will say where.
- **A high flag rate is not failure here.** Flagging 97% of pages the models genuinely couldn't read is correct behavior. The bad outcome would have been a normal flag rate with wrong drug names passing through unreviewed.

What's next

The held-out evaluation: real target-form registers, hand-filled, photographed, never published, created after every model's training cutoff. Those are the numbers that will actually count, and they'll be part 3 of this series.

And the checkbox problem from part 1 still needs its own detection stage, likely using the one model in my tests that produced no hallucinated values.

► The numbers, for engineers (click to expand)

This is part 2 of a series. Part 1: [Confidence Is Theater: What I Learned Building Scribe](#).

Code and methodology: github.com/kod201/scribe · My own benchmark gold set: [sohakanu/scribe-flow-gold](#)

All extraction ran locally on one machine; no page left it.